

Lightweight 3×3 Depthwise Acceleration with Sliding-Window Reuse in a Tightly Coupled RISC-V CFU for INT8 MobileNetV2

line 1: 1st Given Name Surname
line 2: *dept. name of
organization (of Affiliation)*
line 3: *name of organization (of
Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 2nd Given Name Surname
line 2: *dept. name of
organization (of Affiliation)*
line 3: *name of organization (of
Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 3rd Given Name Surname
line 2: *dept. name of
organization (of Affiliation)*
line 3: *name of organization (of
Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 4th Given Name Surname
line 2: *dept. name of
organization (of Affiliation)*
line 3: *name of organization (of
Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 5th Given Name Surname
line 2: *dept. name of
organization (of Affiliation)*
line 3: *name of organization (of
Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 6th Given Name Surname
line 2: *dept. name of
organization (of Affiliation)*
line 3: *name of organization (of
Affiliation)*
line 4: City, Country
line 5: email address or ORCID

Abstract—Tightly coupled custom function units (CFUs) let a RISC-V soft processor accelerate selected TinyML operators at low hardware cost, but the CFU Playground MobileNetV2 design accelerates only 1×1 convolution and leaves depthwise convolution in software. This paper extends that CFU with a lightweight 3×3 depthwise unit and with sliding-window reuse, which keeps the overlapping part of adjacent stride-1 windows inside the CFU. For INT8 MobileNetV2 under TensorFlow Lite Micro on a VexRiscv SoC, the 3×3 CFU reduces whole-model inference cycles by 25.8% relative to the existing 1×1 CFU, and sliding-window reuse raises the reduction to 30.8% while analytically removing 71% of the 3×3 CFU LOAD instructions. All designs meet timing at 75 MHz on an Artix-7 XC7A100T without additional BRAM or DSP slices; the 3×3 unit adds 167 LUTs to the CFU and the reuse logic a further 1,015.

Keywords—RISC-V, custom function unit, depthwise convolution, MobileNetV2, data reuse, FPGA, TinyML

I. INTRODUCTION

Embedded convolutional neural network (CNN) inference must fit within tight memory, latency, and energy budgets [1]. MobileNetV2 [2] was designed for such settings and replaces standard convolutions with cheaper pointwise (1×1) and depthwise (3×3) layers. Even so, one inference still involves a large amount of convolutional computation, which a small soft processor executes slowly.

A lightweight remedy is to extend the processor itself. RISC-V permits application-specific custom instructions, so a custom function unit (CFU) inside the CPU pipeline can execute selected compute-heavy operators while the embedded runtime, such as TensorFlow Lite Micro (TFLM) [3], runs the rest of the model in software. CFU Playground [4] applies this

approach to TinyML on FPGAs and, for MobileNetV2, accelerates the dominant operator, 1×1 pointwise convolution.

Accelerating one operator, however, shifts the bottleneck rather than removing it. Because depthwise convolution and the other operators remain in software, the whole-model speedup reported for CFU Playground is only 3×, far below that of the accelerated operator, and depthwise convolution remains a significant part of the workload. At the other end of the design space, highly specialized accelerators fuse the complete depthwise-separable block and achieve much higher layer-level acceleration with substantially more dedicated hardware. Less characterized is the intermediate design point: extending an existing tightly coupled CFU with depthwise support while keeping its lightweight execution model, and quantifying whether this yields a useful whole-model benefit at reasonable hardware cost.

We investigate this design point by extending the existing MobileNetV2 CFU path with a lightweight 3×3 depthwise CFU and sliding-window data reuse (Fig. 1). The depthwise CFU extends the computational coverage of the hardware; sliding-window reuse reduces redundant data movement between adjacent depthwise windows, which matters because data movement increasingly limits the performance and energy efficiency of ML hardware [5]. Our contributions are:

- A lightweight 3×3 depthwise CFU integrated alongside the existing 1×1 CFU in the RISC-V/TFLM MobileNetV2 execution path.
- Sliding-window data reuse for eligible stride-1 depthwise operations, reducing redundant input movement to the CFU.

- An end-to-end evaluation of INT8 MobileNetV2 inference covering performance, data movement, FPGA resources and timing, and functional correctness.

[FIGURE 1 PLACEHOLDER — single-column, ≈ 2.0 in]

Fig. 1. Overall system architecture: TFLM on VexRiscv, CFU with the existing 1×1 path and the new 3×3 depthwise unit, Artix-7 SoC.

II. BACKGROUND AND RELATED WORK

A. MobileNetV2 and Depthwise Convolution

Each MobileNetV2 inverted residual block applies a 1×1 expansion, a 3×3 depthwise convolution, and a 1×1 projection [2] (Fig. 2). The two layer types load hardware differently. A 1×1 convolution is a matrix–vector product at every pixel: each weight is reused across all pixels and each input across all output channels, which suits wide multiply–accumulate (MAC) units. A depthwise convolution filters each channel with its own 3×3 kernel. It needs far fewer operations, but its inputs cannot be reused across channels [6]. Its main remaining opportunity for input reuse is spatial: adjacent windows of the same channel overlap.

[FIGURE 2 PLACEHOLDER — single-column, ≈ 1.4 in]

Fig. 2. MobileNetV2 inverted residual block and operator mapping: 1×1 expansion/projection → existing 1×1 CFU; 3×3 depthwise → new 3×3 CFU; other operators → software.

B. TFLM and RISC-V CFUs

TFLM executes a quantized model by invoking one kernel per operator [3]. CFU Playground pairs TFLM with a LiteX system-on-chip built around the VexRiscv soft core on an FPGA. Its CFU follows the RISC-V R-type format, receiving two register operands and returning one result, and may keep internal state, so larger operations become short instruction sequences. Acceleration is added per operator: an optimized kernel issues custom instructions for eligible layers, while all other operators stay in software. Its MobileNetV2 CFU, the design we extend, executes 1×1 convolution with four-way INT8 MACs and hardware requantization [4].

C. Related Accelerator Approaches

Prior FPGA work falls into three groups. RISC-V/CFU-based designs keep the processor in control and add hardware incrementally; besides dense operator acceleration, RISC-V extensions have been built for sparse, pruned DNNs [7]. Specialized depthwise-separable accelerators target the block itself: a fused CFU datapath pipelines the expansion, depthwise, and projection layers and achieves high layer-level acceleration with substantially more dedicated hardware [8], and a standalone accelerator packs two 8-bit multiplications into each DSP slice [6]. Arithmetic-level work raises MAC density with mixed DSP/LUT dot-product units [9]; like sparsity, it is complementary to operator coverage.

D. Research Gap and Positioning

CFU Playground demonstrates effective 1×1 acceleration in a tightly coupled RISC-V/TFLM execution model, but its MobileNetV2 CFU leaves depthwise convolution in software; in a keyword-spotting case study, depthwise loops reuse one lane of the convolution MAC unit because a separate depthwise CFU did not fit [4]. The fused design reaches its layer-level gains with a much larger block-level datapath. This leaves an intermediate design point insufficiently characterized: a dedicated but lightweight 3×3 depthwise unit added to the existing CFU, with its whole-model performance, FPGA resource and timing cost, and data movement evaluated together. This work investigates that point and uses sliding-window reuse to address its data-movement side.

III. PROPOSED SYSTEM AND CFU ARCHITECTURE

A. Overall CPU–CFU Architecture

TFLM runs the INT8 MobileNetV2 on the VexRiscv core (Fig. 1). We keep the existing 1×1 CFU datapath unchanged and add a 3×3 depthwise datapath to the same CFU. The CPU invokes both through custom instructions. For 1×1 layers the CFU performs the whole layer. For 3×3 depthwise layers it computes only the nine-term dot product of each window. The CPU handles window addressing, bias, requantization and storing the output, and all remaining operators run in software.

B. 3×3 Depthwise CFU

For one output position of one channel, the 3×3 input window is multiplied element-wise with that channel’s 3×3 weights and the nine products are summed into one value:

$$\begin{array}{ccccc} & & a & b & c \\ d & e & f & \times & \begin{array}{ccc} w_3 & w_4 & w_5 \\ w_6 & w_7 & w_8 \end{array} \\ & & g & h & i \end{array} \rightarrow \Sigma = a \cdot w_0 + b \cdot w_1 + \dots + i \cdot w_8$$

The CFU holds the current input window and weights in registers. The CPU programs the layer offsets once with CONFIGURE and supplies a window with LOAD. RUN then sums the nine products, and GET_RESULT returns the sum. Table I summarizes the interface.

TABLE I. 3×3 DEPTHWISE CFU INSTRUCTIONS

Instruction	Purpose
CONFIGURE	Set input and weight offsets for the layer
LOAD	Store part of the input window and weights (three calls per window)

Instruction	Purpose
RUN	Sum the nine products; the CPU waits for completion
GET_RESULT	Read the 32-bit sum
SHIFT_RIGHT	Move the stored window one position right and insert a new column

C. Pipelined MAC

Our first implementation multiplied nine pairs and summed all products in a single clock cycle, which formed a long critical path. The final design computes and registers the products during LOAD. RUN then reduces them through three registered addition stages (Fig. 3). This shortens the combinational path so the design meets the 75 MHz target.

D. Sliding-Window Data Reuse

At stride 1, neighbouring windows share six of nine inputs:

$$\begin{array}{ccc}
 A & B & C \\
 D & E & F \\
 G & H & I
 \end{array}
 \rightarrow
 \begin{array}{ccc}
 B & C & J \\
 E & F & K \\
 H & I & L
 \end{array}$$

Instead of loading the complete next window, the CFU keeps the current one and receives only the new rightmost column (J, K, L). The stored weights are reused for the same channel. SHIFT_RIGHT moves the stored window one position to the right and inserts that column. The next RUN first recomputes the nine products from the shifted window, which costs one extra cycle. The first window uses LOAD $\times 3 \rightarrow$ RUN $\rightarrow$ GET_RESULT, and each next window uses SHIFT_RIGHT $\rightarrow$ RUN $\rightarrow$ GET_RESULT. Reuse applies only to stride-1 windows that lie fully inside the input. Stride-2 layers and border windows that overlap padding do not use it.

[FIGURE 3 PLACEHOLDER — single-column, ≈ 1.8 in]

Fig. 3. 3 $\times$ 3 depthwise CFU datapath and sliding-window reuse. LOAD fills the input/weight windows and product registers; SHIFT_RIGHT inserts one new column (J, K, L) and retains six inputs and all weights; RUN drives the three-stage adder pipeline.

E. Software/Hardware Integration

The depthwise kernel uses the CFU for INT8 3 $\times$ 3 layers with stride 1 or 2, dilation 1, depth multiplier 1 and zero weight offset. All other cases use the reference software kernel. For eligible stride-1 regions, the software processes consecutive horizontal positions of the same output channel together. This keeps that channel's weights in the CFU while the input window moves across the row. The first window of a row loads the complete window, and the following windows use SHIFT_RIGHT. After each GET_RESULT, the CPU applies bias, requantization and clamping and stores the INT8 output.

IV. EXPERIMENTAL METHODOLOGY

A. Workload and Platform

All configurations run the same INT8 MobileNetV2 (width 0.35) on the same embedded 224 $\times$ 224 image (Table II). Resources and timing come from Vivado 2024.1 builds of the SoC.

TABLE II. PLATFORM AND CONFIGURATIONS

Item	Value
Model	MobileNetV2 a0.35, mobilenetv2_a035_224_int8.tflite
Input / precision	224 $\times$ 224 $\times$ 3 UINT8 image; INT8 internal; FLOAT32 output
Processor	VexRiscv (LiteX/CFU-Playground SoC)
FPGA / clock	XC7A100T (Nexys4 DDR target); 75 MHz system clock
Tools	TFLM, Amaranth, Vivado 2024.1
Cycle-count platform	[REQUIRES VERIFICATION: board or simulation]
C1	Existing 1 $\times$ 1 CFU (baseline)
C2	C1 + 3 $\times$ 3 depthwise CFU
C3	C2 + sliding-window reuse

B. Experimental Configurations

The three configurations form an incremental ablation. C1 is the baseline, C2 isolates the contribution of the 3 $\times$ 3 CFU, and C3 isolates the contribution of window reuse. They use the same model, input and profiler and differ only in the accelerator build.

C. Evaluation Metrics

We measure end-to-end inference cycles as the overall performance outcome, and per-operator profiler ticks to locate where time is spent. CFU instruction counts quantify data movement; they reflect instruction traffic, not time. Post-place LUT, FF, BRAM and DSP usage give the hardware cost, and post-route WNS, TNS and failing endpoints give timing. Unit-test results, top-1 class and score assess functional correctness.

D. Verification

The CFU is verified by 25 gateway unit tests in simulation, covering the 3 $\times$ 3 MAC, window shifting and the instruction interface. A dedicated build adds per-layer checksums and instruction counters for comparison with the reference kernel. End-to-end, C1–C3 run the same input and their outputs are compared.

V. RESULTS AND DISCUSSION

A. End-to-End Performance

TABLE III. PERFORMANCE (ONE RUN PER CONFIGURATION)

Metric	C1	C2	C3
Inference cycles	828,991,457	615,396,021	573,258,998
Speedup vs. C1	1.00 $\times$	1.35 $\times$	1.45 $\times$
Cycle reduction vs. C1	—	25.8%	30.8%
Depthwise share of profile	70.4%	60.1%	55.3%
LOAD instructions	—	2,529,552	726,864
SHIFT_RIGHT instructions	—	0	600,896

Moving 3×3 depthwise computation from software into the CFU (C2) gives the main improvement over C1. Reusing overlapping windows (C3) improves cycles further. Together the proposed additions reduce inference cycles by 30.8% relative to C1. Depthwise layers still account for over half of the profile in C3.

B. Sliding-Window Effect

Window reuse reduces LOAD instructions by about 71% (Table III). The end-to-end gain is much smaller, because every window still needs RUN, GET_RESULT and software-side processing.

C. FPGA Resources and Timing

TABLE IV. POST-PLACE RESOURCES AND POST-ROUTE TIMING (75 MHz)

Metric	C1	C2	C3
LUT (total / CFU)	6,187 / 1,511	6,865 / 1,678	8,017 / 2,693
FF (total / CFU)	4,504 / 1,081	4,898 / 1,475	5,080 / 1,657
RAMB36 / RAMB18	15 / 33	15 / 33	15 / 33
DSP	8	8	8
WNS (ns)	+0.962	+0.372	+0.322
Failing endpoints	0	0	0

The 3×3 CFU adds a modest LUT/FF cost to the CFU (Table IV). Sliding-window reuse costs more logic than the 3×3 unit itself. No additional BRAM or DSP is needed in any configuration. All three designs meet the 75 MHz target with positive slack. We did not measure Fmax.

D. Functional Correctness

All 25 gateway unit tests pass (simulation). C1–C3 process the same input and return the same top-1 class (index 64) and score, with matching output statistics. A full output checksum is printed only in the C3 log, so bit-exact equality across configurations is not demonstrated. Results of the checksum build are not reported here. We make no accuracy claim, since only one image was used.

E. Overall Discussion

Adding the 3×3 depthwise CFU gives the main performance improvement at a small hardware cost. Sliding-window reuse gives an additional improvement by reducing repeated input transfer, but needs more logic for a smaller cycle gain. The complete design keeps BRAM and DSP usage unchanged and meets timing at 75 MHz. The remaining runtime is dominated by software work around the CFU and by operators that stay on the CPU.

VI. LIMITATIONS AND FUTURE WORK

Each configuration was measured once, so run-to-run variation is not quantified, and the cycle-count logs do not state whether they were obtained on the board or in simulation. No CPU-only baseline was measured on the same setup, so speedups are reported relative to the existing 1×1 CFU. CFU instruction counts are derived analytically, and bit-exact output equality across configurations is not yet demonstrated. Future work will repeat the measurements and reduce the per-window software work that remains around the CFU.

VII. CONCLUSION

We extended the CFU Playground MobileNetV2 CFU with a lightweight 3×3 depthwise unit and sliding-window reuse, keeping the existing 1×1 datapath and execution model unchanged. The 3×3 unit provides most of the gain, a 25.8% reduction in whole-model cycles for a small LUT/FF cost and no additional BRAM or DSP. Reuse removes most redundant input loads but adds a smaller cycle gain for more logic, consistent with the remaining runtime being dominated by software work around the CFU and by operators that stay on the CPU.

REFERENCES

- [1] C. Banbury *et al.*, “MLPerf Tiny benchmark,” 2021, arXiv:2106.07597.
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018.
- [3] R. David *et al.*, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” *Proc. Mach. Learn. Syst.*, vol. 3, pp. 800–811, 2021.
- [4] S. Prakash *et al.*, “CFU Playground: Full-stack open-source framework for tiny machine learning (TinyML) acceleration on FPGAs,” 2023, arXiv:2201.01863v3.
- [5] M. S. Vahdatpour and Y. Zhang, “Energy-efficient software–hardware co-design for machine learning: From TinyML to large language models,” 2026, arXiv:2603.23668.
- [6] X. Li, H. Huang, Y. Liu, X. Hu, and X. Xiong, “A digital signal processor-efficient accelerator for depthwise separable convolution,” *Electron. Lett.*, vol. 58, no. 7, pp. 271–273, Mar. 2022, doi: 10.1049/ell2.12435.
- [7] M. Sabih, A. Karim, J. Wittmann, F. Hannig, and J. Teich, “Hardware/software co-design of RISC-V extensions for accelerating sparse DNNs on FPGAs,” in *Proc. Int. Conf. Field-Programmable Technol. (FPT)*, 2024.
- [8] M. Yildirim and O. Ozturk, “RISC-V based TinyML accelerator for depthwise separable convolutions in edge AI,” 2025, arXiv:2511.21232.
- [9] M. Véstias, R. Duarte, J. T. de Sousa, and H. Neto, “Parallel dot-products for deep learning on FPGA.” [REQUIRES VERIFICATION: venue and year]